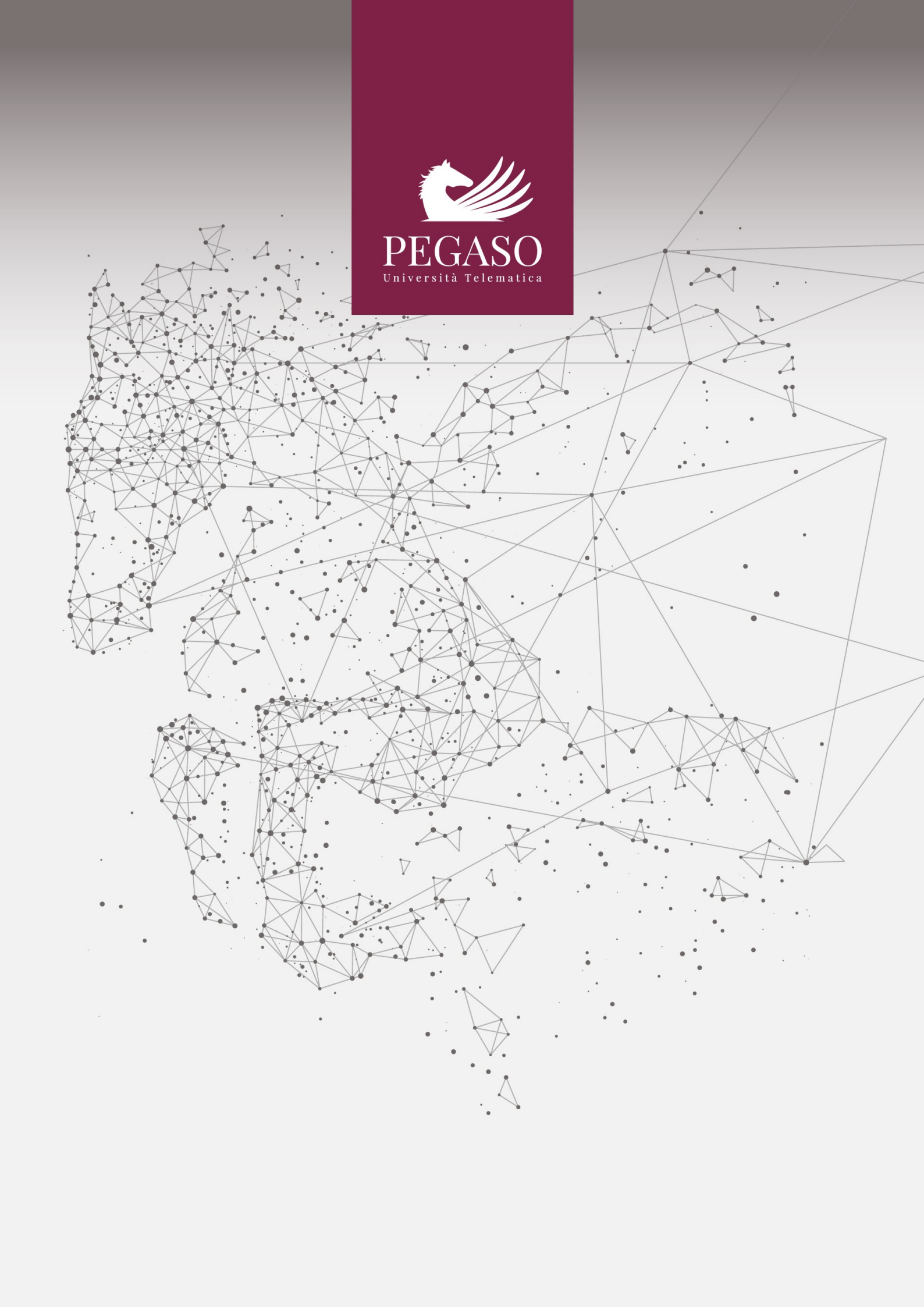

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE A HIGH PERFORMANCE COMPUTING 3

2. CLUSTER COMPUTING.....4

3. GRID COMPUTING 6

BIBLIOGRAFIA 9

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione a High Performance Computing

La categoria dei sistemi distribuiti utilizzati per le operazioni di calcolo ad alte prestazioni è importante. Si può, grosso modo, distinguere tra due sottogruppi all'interno del gruppo principale.

Nel cluster computing, l'hardware sottostante è costituito da un numero di workstation o personal computer che sono quasi identici tra loro e sono collegati tra loro da una rete locale ad alta velocità. Inoltre, lo stesso sistema operativo viene utilizzato da ogni singolo nodo.

Quando si tratta di grid computing, lo scenario assume una piega completamente diversa. Questo sottogruppo comprende sistemi distribuiti, che sono spesso costruiti come se fossero una federazione di sistemi informatici. Ogni sistema della federazione può appartenere a un dominio amministrativo separato e possono avere hardware, software e tecnologie di rete molto diversi implementati.

Se si fa riferimento a come è fatto il grid computing, il prossimo passo che più sensato è quello di esternalizzare l'intera infrastruttura necessaria per le applicazioni che richiedono molta potenza di elaborazione. Fornire la capacità di sviluppare dinamicamente un'infrastruttura e assemblare ciò che è richiesto dai servizi esistenti è al centro di ciò che è il cloud computing. Il cloud computing, d'altra parte, rappresenta molto più che mettere a disposizione un gran numero di risorse, a differenza del grid computing, che è strettamente connesso con il calcolo ad alte prestazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Cluster computing

Il rapporto prezzo/prestazioni dei personal computer e delle workstation è aumentato nel corso del tempo, il che ha contribuito all'aumento della popolarità dei sistemi di elaborazione cluster. A un certo punto, è diventato sia finanziariamente che tecnicamente vantaggioso creare dei supercomputer utilizzando la tecnologia standard semplicemente collegando un insieme di computer relativamente semplici in una rete ad alta velocità. Il cluster computing viene praticamente sempre utilizzato per la programmazione parallela, in cui un singolo programma (ad alta intensità di risorse) viene eseguito in parallelo su numerosi computer. Questo tipo di elaborazione è noto come "clustering".

I cluster Beowulf, che sono equipaggiati con Linux e hanno una configurazione generale che è mostrata in Figura 1, sono un esempio di un tipo di computer cluster che hanno trovato un uso piuttosto diffuso. Ogni cluster è costituito da un gruppo di nodi di calcolo, accessibili e gestiti tramite l'utilizzo di un singolo nodo master. Il master gestisce spesso il processo di allocazione dei nodi a un determinato programma parallelo, mantiene una coda batch di attività che sono state inviate e offre un'interfaccia per gli utenti del sistema. Di conseguenza, il middleware necessario per l'esecuzione dei programmi e la gestione del cluster è effettivamente gestito dal master, mentre i nodi di calcolo sono dotati di un sistema operativo standard che ha funzioni middleware tipiche aggiunte ad esso per la comunicazione, l'archiviazione, la tolleranza di errore e così via. I nodi di calcolo, ad eccezione del nodo master, sembrano essere estremamente simili tra loro.

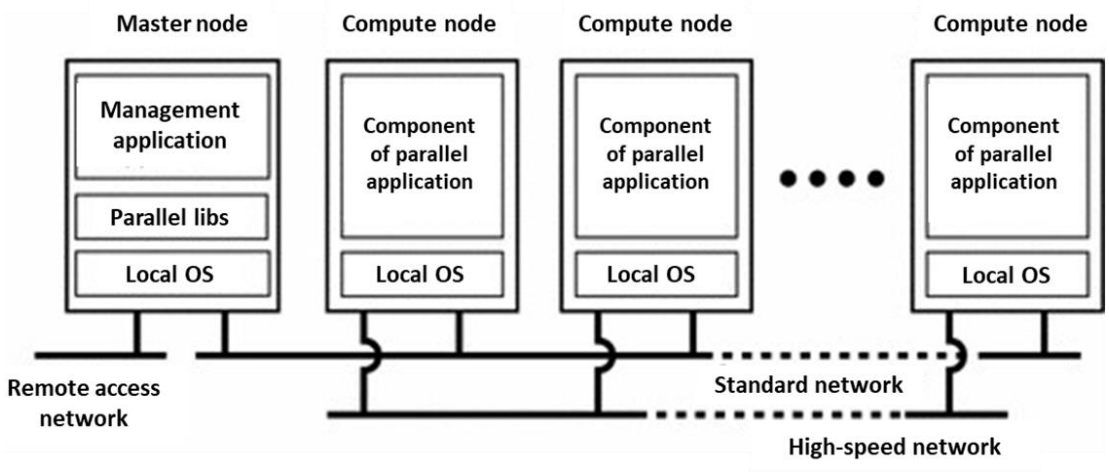


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2016). A brief introduction to distributed systems. Computing

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il sistema MOSIX adotta un approccio ancora più simmetrico di quello sopra descritto. MOSIX fornisce un'immagine a sistema singolo di un cluster, il che significa che per un processo, un computer cluster sembra essere un singolo computer e fornisce il massimo livello di trasparenza di distribuzione possibile. Come è stato notato prima, è impossibile fornire un tale quadro in tutte le condizioni. Nel caso di MOSIX, l'elevato grado di trasparenza si ottiene consentendo ai processi di spostarsi tra i nodi che compongono il cluster in modo dinamico e preventivo. Un utente è in grado di avviare un'applicazione su qualsiasi nodo (che viene indicato come nodo principale) e, successivamente, il programma è in grado di migrare in modo trasparente ad altri nodi al fine di utilizzare efficacemente le risorse. Ciò è reso possibile tramite la migrazione dei processi.

I sistemi ibridi includono il middleware partizionato funzionalmente tra vari nodi. Il vantaggio di tale separazione è evidente: avere nodi di calcolo dotati di sistemi operativi specializzati e leggeri si tradurrà quasi certamente in prestazioni ottimali per le applicazioni che richiedono molta potenza di calcolo. Allo stesso modo, la funzionalità di archiviazione può quasi certamente essere gestita correttamente da altri nodi che sono stati specificamente progettati, come file server e server di directory. Lo stesso vale per molti altri tipi di servizi middleware specializzati, come la gestione dei lavori, i servizi di database e forse anche la connettività Internet generale ad altri tipi di servizi esterni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Grid computing

L'omogeneità è una delle caratteristiche che definiscono l'approccio convenzionale al calcolo in cluster. La maggior parte delle volte, i computer che compongono un cluster sono quasi identici tra loro; seguono lo stesso sistema operativo e sono tutti collegati alla stessa rete. D'altra parte, come abbiamo appena visto, c'è stato un movimento verso architetture più ibride in cui i singoli nodi sono costruiti appositamente per svolgere un particolare insieme di responsabilità.

Nei sistemi di grid computing, questa variazione è considerevolmente più diffusa poiché non vengono fatte ipotesi a priori sulla compatibilità di hardware, sistemi operativi, reti, domini amministrativi o regole di sicurezza, tra le altre cose. Un aspetto cruciale di un sistema informatico adibito a grid computing è che le risorse di diverse organizzazioni sono raggruppate insieme al fine di facilitare la cooperazione di un gruppo di individui provenienti da vari stabilimenti, dando così origine al concetto di una federazione di sistemi. Un'organizzazione virtuale funge da veicolo per la realizzazione di tale partnership. Ai processi associati alla stessa organizzazione virtuale viene concessa l'autorizzazione per accedere alle risorse rese disponibili a tale organizzazione. In genere, le risorse sono costituite da server di elaborazione (inclusi i supercomputer, che possono essere distribuiti come computer cluster), strutture di archiviazione e sistemi di gestione di database. Oltre a questo, possono essere fornite anche apparecchiature di rete specializzate come telescopi, sensori e altri elementi simili.

A causa della natura del grid computing, la maggior parte del software sviluppato per renderlo una realtà ruota attorno alla fornitura di accesso alle risorse da vari domini amministrativi e questo accesso è limitato agli utenti e alle applicazioni che sono membri di una particolare organizzazione virtuale. Per questo motivo, l'attenzione è spesso posta sulle difficoltà architettoniche. La Figura 2 illustra un'architettura che è stata sviluppata per la prima volta all'inizio degli anni 2000, e questo design continua a servire come base per molti sistemi di grid computing.

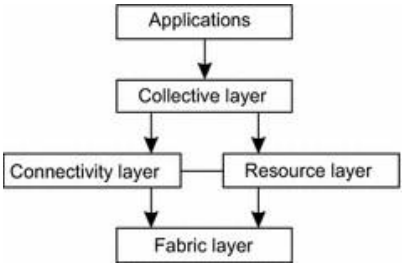


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2016). A brief introduction to distributed systems. Computing

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'architettura è divisa in quattro strati distinti. Il livello inferiore di base (Fabric layer) funge da interfaccia per le risorse locali disponibili in una determinata posizione. Si noti che queste interfacce sono state adattate specificamente in modo che i membri di un'organizzazione virtuale possano condividere le risorse tra loro. Nella maggior parte dei casi, includeranno metodi per ottenere informazioni sullo stato corrente e sulle capacità di una risorsa, oltre alle funzioni per la gestione della risorsa stessa (ad esempio, il blocco delle risorse).

Il livello di connessione (Connectivity layer) è costituito da diversi protocolli di comunicazione utilizzati per facilitare le transazioni di rete che includono l'uso di una serie di risorse diverse. I protocolli sono necessari, ad esempio, per spostare i dati da una risorsa all'altra o anche solo per accedere a una risorsa quando si trova in un luogo lontano. Inoltre, l'autenticazione degli utenti e delle risorse verrà gestita tramite protocolli di sicurezza che verranno inclusi nel livello di connessione. È importante tenere presente che in molti casi, gli utenti umani effettivi non vengono verificati. Piuttosto, i programmi che operano per conto degli utenti vengono autenticati. In questo contesto, la delega dei privilegi di un utente a un programma è una funzione cruciale che deve essere gestita dal livello di connessione.

Il livello di risorse (Resource Layer) ha il compito di gestire una singola risorsa. Effettua chiamate dirette alle interfacce rese accessibili dal Fabric Layer e utilizza le funzionalità fornite dal livello di connettività. Ad esempio, i metodi per raccogliere informazioni di configurazione su una particolare risorsa o, più in generale, funzioni per eseguire determinate attività come l'avvio di un processo o la lettura dei dati saranno forniti da questo livello quando viene implementato. L'autenticazione eseguita come parte del livello di connessione sarà quindi invocata dal livello di risorsa poiché è considerato responsabile del controllo degli accessi a quel livello.

Il livello collettivo (Collective Layer) viene immediatamente dopo nella gerarchia architetturale presa in esame. Si occupa della gestione dell'accesso a molte risorse e spesso include servizi per l'identificazione delle risorse, l'allocazione e la programmazione delle attività tra varie risorse, la replica dei dati e altri argomenti correlati. Il livello collettivo, a differenza del livello di connettività e del livello di risorse, può essere costituito da un'ampia varietà di protocolli, riflettendo l'ampia gamma di servizi che può fornire a un'organizzazione virtuale. Sia il livello di connettività che il livello di risorsa sono costituiti da un insieme relativamente piccolo e standardizzato di protocolli.

Il terzo e ultimo livello è il livello dell'applicazione (Application Layer). Questo livello è costituito da applicazioni eseguite all'interno di un'organizzazione virtuale e utilizzano l'ambiente di grid computing.

Nella maggior parte dei casi, i livelli collettivi, di connessione e di risorse costituiscono il nucleo di quello che viene spesso definito il livello middleware del grid computing. Questi livelli lavorano insieme per fornire agli utenti l'accesso e l'amministrazione delle risorse che potrebbero trovarsi in diverse posizioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il concetto di sito, noto anche come unità amministrativa, è ampiamente utilizzato nel grid computing. La crescente prevalenza di tale nozione ha determinato la progressiva transizione verso un'architettura orientata ai servizi che si è verificata di recente.

In questa architettura, i siti Web forniscono l'accesso ai diversi livelli utilizzando un set di servizi Web. Ciò ha portato in questo modo alla creazione di un design alternativo noto come Open Grid Services Architecture (OGSA). OGSA è stato sviluppato sulla base dei concetti di inizio anni 2000, ed il fatto che sia stato sottoposto a una procedura di standardizzazione lo ha reso complesso. In generale, le implementazioni OGSA aderiscono agli standard per i servizi web.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- M. van Steen and A.S. Tanenbaum, A brief introduction to distributed systems. Computing, 2016.
- Amar L., Barak A., and Shiloh A. The MOSIX Direct File System Access Method for Supporting Scalable Cluster File Systems. Cluster Computing, 7(2):141–150, Apr. 2004.
- Foster I., Kesselman C., and Tuecke S. The Anatomy of the Grid, Enabling Scalable Virtual Organizations. Journal of Supercomputer Applications, 15(3):200–222, 2001.
- Foster I. and others. The Open Grid Services Architecture, Version 1.5. GGF Informational Document GFD-I.080, June 2006.
- Engelmann C., Ong H., and Scott S. Middleware in Modern High Performance Computing System Architectures. In International Conference on Computational Science, volume 4488 of Lecture Notes in Computer Science, pages 784–791, Berlin, May 2007.
- Joseph J., Ernest M., and Fellenstein C. Evolution of grid computing architecture and grid adoption models. IBM Systems Journal, 43(4):624–645, Apr. 2004.
- Lottiaux R., Gallard P., Vallee G., and Morin C. OpenMosix, OpenSSI and Kerrighed: A Comparative Study. In 5th International Symposium on Cluster Computing and the Grid, pages 1016–1023, Los Alamitos, CA., May 2005. IEEE Computer Society Press.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).